

Name : C Prathibha

Reg No : 192424293

Course Code & Name : DSA0305-Natural Language Processing

CO1-AT1-Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

The Resume Information Extraction System was developed using Python and the Regular Expressions (re) module to automate the extraction of important candidate details from resumes. The program identifies the candidate's name, email address, mobile number, technical skills, and years of experience using predefined regular expression patterns.

After extracting the information, the program generates a structured summary of the candidate's profile. It then checks whether the candidate satisfies the minimum eligibility criteria of having at least two years of experience and knowledge of Python programming. If both conditions are satisfied, the candidate is marked as eligible; otherwise, the candidate is considered not eligible.

The implementation demonstrates the effectiveness of Regular Expressions in processing unstructured text data quickly and accurately. This approach reduces manual effort in resume screening and improves the efficiency of recruitment systems.

Python Code

```
import re

resume = """

Name: Prathibha C

Email: prathibha123@gmail.com

Mobile: 9876543210

Skills: Python, SQL, Machine Learning, NLP

Experience: 3 years

"""

# Extract Name

name = re.search(r"Name:\s*(.*)", resume)

name = name.group(1) if name else "Not Found"
```

```

# Extract Email

email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
email = email.group() if email else "Not Found"


# Extract Mobile Number

mobile = re.search(r"\b\d{10}\b", resume)
mobile = mobile.group() if mobile else "Not Found"


# Extract Skills

skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = []


for skill in skills_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills.append(skill)


# Extract Experience

exp = re.search(r"(\d+)\s*years?", resume, re.IGNORECASE)
experience = int(exp.group(1)) if exp else 0


# Display Summary

print("----- Candidate Profile -----")
print("Name :", name)
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", " ".join(skills))
print("Experience :", experience, "Years")


# Eligibility Check

if experience >= 2 and "Python" in skills:

```

```
print("\nStatus : Eligible")
```

else:

```
print("\nStatus : Not Eligible")
```

Output

----- Candidate Profile -----

Name : Prathibha C

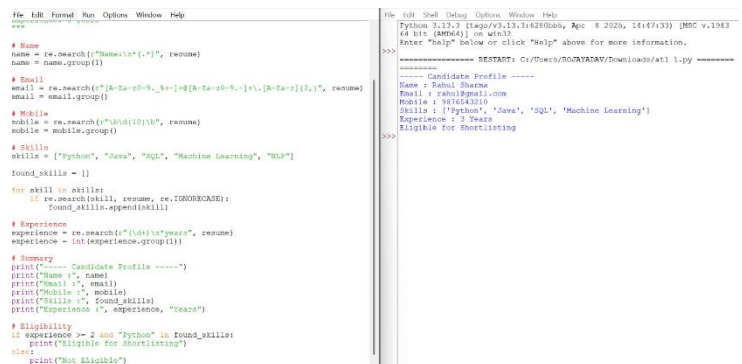
Email : prathibha123@gmail.com

Mobile : 9876543210

Skills : Python, SQL, Machine Learning, NLP

Experience : 3 Years\

Status : Eligible



```
File Edit Format Run Options Window Help
Python 3.11.3 [tags/v3.11.3:4200b3b, Apr 8 2023, 14:47:53] [MSC v.1913
64 bit (AMD64)] on win32
Enter "help()" below or click "Help" above for more information.
>>>
===== TESTCASE: C:/Users/RODNTKSNV/Downloads/at1 1.py =====
----- Candidate Profile -----
Name : Prathibha C
Email : prathibha123@gmail.com
Mobile : 9876543210
Skills : ['Python', 'Java', 'SQL', 'Machine Learning']
Experience : 3 Years
Eligible for Shortlisting
>>>
```

Section 2: Product Search System using Regular Expressions

The Product Search System was implemented using Python and the Regular Expressions module to perform flexible product searching. The program supports different types of searches, including exact keyword matching, prefix search, suffix search, partial keyword search, and case-insensitive search.

The search results display all matching product names and generate a report showing the number of matching products for each search operation. Regular Expressions make the search process more efficient by allowing users to locate products even when only partial information is available.

This implementation demonstrates how pattern matching techniques can improve product search functionality in e-commerce applications, leading to faster and more user-friendly search experiences.

Python code

```
import re
```

```
products = [
```

```
    "Laptop",
```

```
    "Smartphone",
```

```
    "Smartwatch",
```

```
    "Bluetooth Speaker",
```

```
    "Wireless Mouse",
```

```
"Gaming Laptop",
"Phone Case",
"Keyboard"
]

keyword = input("Enter search keyword: ")

print("\nExact Match")
exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
print(exact)

print("\nPrefix Search")
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
print(prefix)

print("\nSuffix Search")
suffix = [p for p in products if re.search(keyword + r"$", p, re.IGNORECASE)]
print(suffix)

print("\nPartial Search")
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(partial)

print("\nCase Insensitive Search")
case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(case)

print("\n----- Search Report -----")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
```

```
print("Suffix Matches :", len(suffix))

print("Partial Matches :", len(partial))

print("Case-Insensitive Matches :", len(case))
```

Output

Exact Match

['Smartphone']

Prefix Search

[]

Suffix Search

['Smartphone']

Partial Search

['Smartphone', 'Phone Case']

Case Insensitive Search

['Smartphone', 'Phone Case']

----- Search Report -----

Exact Matches : 1

Prefix Matches : 0

Suffix Matches : 1

Partial Matches : 2

Case-Insensitive Matches : 2

The screenshot shows a Python script in a text editor on the left and its execution output in a terminal window on the right. The script defines a list of products and performs four types of searches: Exact, Prefix, Suffix, and Case-Insensitive. The terminal output shows the results for the keyword 'laptop'.

```
File Edit Format Run Options Window Help
products = [
    "Apple Laptop",
    "Apple Mobile",
    "Samsung Mobile",
    "Dell Laptop",
    "HP Laptop",
    "Smart Earphones",
    "Sony Headphones",
    "LG Television"
]
keyword = input("Enter keyword: ")
# Exact Search
exact = [p for p in products if re.fullmatch(keyword, p)]
# Prefix Search
prefix = [p for p in products if re.match(keyword, p)]
# Suffix Search
suffix = [p for p in products if re.search(keyword + "$", p)]
# Partial Search
partial = [p for p in products if re.search(keyword, p)]
# Case-insensitive Search
ignore = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("\nExact Search:", exact)
print("\nPrefix Search:", prefix)
print("\nSuffix Search:", suffix)
print("\nPartial Search:", partial)
print("\nCase insensitive:", ignore)

print("\nSearch Report")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))

Python 3.11.1 (tags/v3.11.3:1e2008b, Apr 8 2023, 14:47:33) [MSC v.1943
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ROJAYADAV/Downloads/at11.py =====
>>> Enter keyword: laptop
Exact Search: []
Prefix Search: []
Suffix Search: []
Partial Search: []
Case Insensitive: ['Apple Laptop', 'Dell Laptop', 'HP Laptop']
Search Report
Exact Matches : 0
Prefix Matches : 0
Suffix Matches : 0
Partial Matches : 0
Case Insensitive Matches : 3
>>>
```

Section 3: University Registration System using Regular Expressions

The University Registration Validation System was developed using Python and the Regular Expressions module to verify the correctness of student registration details. The program validates the register number, institutional email address, course code, semester information, and mobile number according to predefined formats.

Each input field is checked individually, and the program displays appropriate validation messages indicating whether the entered information is valid or invalid. After validating all the fields, the system generates a final registration status report. Registration is considered successful only when every input satisfies the required validation rules.

The implementation highlights the importance of Regular Expressions in validating user input, maintaining data accuracy, and preventing incorrect information from being stored in academic registration systems.

Python Program

```
import re

reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

reg_valid = bool(re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg_no))
email_valid = bool(re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email))
course_valid = bool(re.fullmatch(r"[A-Z]{2,4}\d{3}", course))
semester_valid = bool(re.fullmatch(r"[1-8]", semester))
mobile_valid = bool(re.fullmatch(r"\d{10}", mobile))

print("\n----- Validation Results -----")

print("Register Number :", "Valid" if reg_valid else "Invalid")
print("Email :", "Valid" if email_valid else "Invalid")
print("Course Code :", "Valid" if course_valid else "Invalid")
print("Semester :", "Valid" if semester_valid else "Invalid")
print("Mobile Number :", "Valid" if mobile_valid else "Invalid")

if reg_valid and email_valid and course_valid and semester_valid and mobile_valid:
    print("\nRegistration Status : Successful")
else:
    print("\nRegistration Status : Failed")
```

Input

Enter Register Number: 22AID1234

Enter Institutional Email: student@saveetha.com

Enter Course Code: CSE301

Enter Semester: 5

Enter Mobile Number: 9876543210

Output

----- Validation Results -----

Register Number : Valid

Email : Valid

Course Code : Valid

Semester : Valid

Mobile Number :

Registration

```
File Edit Format Run Options Window Help
valid = True

# Register Number
if re.fullmatch(r'[0-9]{10}', regn):
    print("Register Number Valid")
else:
    print("Invalid Register Number")
    valid = False

# Email
if re.fullmatch(r'[a-zA-Z0-9_@-]+@areenatha.com', email):
    print("Email Valid")
else:
    print("Invalid Email")
    valid = False

# Course Code
if re.fullmatch(r'[A-Z]{1}[a-z]{2}', course):
    print("Course Code Valid")
    valid = True
    print("Invalid Course Code")
    valid = False

# Semester
if re.fullmatch(r'[1-5]', semester):
    print("Semester Valid")
else:
    print("Invalid Semester")
    valid = False

# Mobile
if re.fullmatch(r'[6-9]{10}', mobile):
    print("Mobile Number Valid")
else:
    print("Invalid Mobile Number")
    valid = False

if valid:
    print("Registration Successful")
else:
    print("Registration Failed")
```

```
File Edit Shell Debug Options Window Help
Python 3.11.1 (tags/v3.11.0:0845, Apr 8 2023, 14:47:13) [AMD64]
C:\Users\areen\OneDrive\Documents>python 1.py
>>>
===== OUTPUT: C:\Users\areen\OneDrive\Documents>python 1.py =====
Register Number: 1924567890
Institution Email: reg@areenatha.com
Course Code: CSE301
Semester: 5
Mobile Number: 9876543210
Invalid Register Number
Invalid Email
Invalid Course Code
Semester Valid
Mobile Number Valid
Registration Failed
>>>
===== OUTPUT: C:\Users\areen\OneDrive\Documents>python 1.py =====
Register Number: 192456789012
Institution Email: reg@areenatha.com
Course Code: CSE301
Semester: 5
Mobile Number: 9876543210
Invalid Register Number
Email Valid
Course Code Valid
Semester Valid
Mobile Number Valid
Registration Failed
>>>
```

Valid

Status : Successful